MFE SERVICES SDN. BHD.

MFE Inspection System

Office PC Technical Guide

Running the system · Manager accounts · Backup and recovery · Security details

App address: https://app.mfets.com
Version: 22 October 2026

For the person who supports the office computer. Confidential: it describes how the system is run and protected. Never write passwords in it.

Contents

1. Running the System 3
1.1 What runs 3
1.2 Where things are kept 3
1.3 Start, stop, check 4
1.4 After a reboot or power cut 4
1.5 Settings in `.env` 4
1.6 Installing an update 4
1.7 Rolling back 5
1.8 Never 5
2. Accounts (Manager accounts and recovery) 6
2.1 Role names on screen and in the database 6
2.2 Create a Manager 6
2.3 Reset a Manager password (also re-activates the account) 6
2.4 Find Manager usernames 6
2.5 Retire a Manager who leaves 6
2.6 Technician and Admin accounts 7
3. Backup and Recovery 8
3.1 What one backup set contains 8
3.2 The daily scheduled task (set up once) 8
3.3 Retention 9
3.4 Restore drill (every 3 months; never touches live data) 9
3.5 Rebuild after the PC dies, or restore a damaged database 9
3.6 Never 10
4. Security Details 11
If a secret leaks 11

1. Running the System

For the support person / IT. All commands are **PowerShell**, run in `C:\InspectionSystem\app` (the folder with `docker-compose.yml`), with Docker Desktop showing **Engine running**. This installation uses the Cloudflare Tunnel, so include `--profile tunnel` on `up`, `ps` and `stop`.

1.1 What runs

Service	Purpose	Reachable from outside?
`proxy` (Caddy)	Serves the web app and passes `/api` to the API	Only through the tunnel (also on the PC's own ports 80/443 on the LAN)
`api`	Accounts, sync, approvals, reports and PDFs	No
`reminder-worker`	Sends reminder e-mails (checks every 15 minutes)	No (outbound to Gmail only)
`postgres`	The database	**Never**
`cloudflared`	Cloudflare Tunnel connector (outbound only)	Outbound only

1.2 Where things are kept

What	Location	In the daily backup?
Database	Docker volume `inspection_pwa_postgres_data`	Yes (`Backup-All.ps1`)
Photos	`C:\InspectionSystem\runtime\uploads`	Yes
Letterhead, uploaded logo, e-mail encryption key	`C:\InspectionSystem\runtime\operational\`	Yes
Backups	`C:\InspectionSystem\runtime\backups\` (+ the off-PC place)	n/a
Logs	`C:\InspectionSystem\runtime\logs\api`, `...\logs\proxy`, `backups\backup-log.txt`	No
Web app versions (current and previous)	`C:\InspectionSystem\runtime\releases`	No (rebuilt from the code)
Secrets and settings	`C:\InspectionSystem\app\.env`	**No. A copy is kept in the client's password manager**
The application code	`C:\InspectionSystem\app` (branch `client-release`)	No (comes from the release repository)

1.3 Start, stop, check

```
docker compose --profile tunnel ps                    # all Up; postgres (healthy)
docker compose --profile tunnel up -d                 # start (also after a manual stop)
docker compose --profile tunnel stop                  # stop without deleting anything
docker compose logs api --tail=100
docker compose logs cloudflared --tail=50
docker compose logs reminder-worker --tail=50
curl.exe -s -H "Host: app.mfets.com" http://localhost/api/health      # local check, no internet
needed
.\scripts\Health-Check.ps1 -BaseUrl https://app.mfets.com -StrictCertificateCheck   # through the
internet
```

Healthy: `{"status":"ok","service":"inspection-api",...}`.

1.4 After a reboot or power cut

Docker Desktop starts only after Windows is signed in. After sign-in, wait for **Engine running**; the containers restart by themselves (`restart: unless-stopped`). If any shows `Exited`: `docker compose --profile tunnel up -d`.

1.5 Settings in `.env`

Edit `.env`, then `docker compose --profile tunnel up -d` to apply. Never put real values in `.env.example`, a screenshot or chat, and never run `docker compose config` without `--quiet` on a shared screen (it prints the secrets).

Setting	Meaning
`PUBLIC_HOSTNAME`	`http://app.mfets.com` (the `http://` is required with the tunnel; without it the browser loops)
`CLOUDFLARE_TUNNEL_TOKEN`	Tunnel token from Cloudflare Zero Trust → Networks → Tunnels
`POSTGRES_PASSWORD` / `DATABASE_URL`	Must match. Postgres keeps the password it was created with; do not change it casually
`SESSION_DURATION_HOURS`	How long a sign-in lasts (default 12)
`INSPECTION_*_PATH`	Runtime folders; keep the defaults under `C:\InspectionSystem\runtime`

1.6 Installing an update

The PC tracks the `client-release` branch of the release repository, which contains only what the client needs.

1. Ask technicians to sync everything (**All submitted changes synced**).
2. `.\scripts\Backup-All.ps1 -CopyTo '<off-PC folder>'` → must end with **Backup finished OK**.
3. Note the current version: `git log -1 --format="%h %s"`.
4. Get the new version and rebuild:

   ```
   git pull
   docker compose --profile tunnel up -d --build
   ```

5. `docker compose --profile tunnel ps`, `docker compose logs api --tail=50`, health check, then sign in as each role. Compare `.env` with `.env.example` for new settings.
6. Phones receive the new app in the background and show **Update now**.

Database changes (migrations) run automatically when the API starts and only move forward.

1.7 Rolling back

Problem	Roll back
Only the web app looks wrong	Read the two retained IDs in `C:\InspectionSystem\runtime\releases\release-order`, then `$env:INSPECTION_WEB_ACTIVE_RELEASE = "sha256-<previous>"; docker compose up -d --no-deps --force-recreate proxy`. Remove the variable and recreate the proxy to return. No data is touched.
The new version misbehaves and added no database change	`git checkout <previous commit>` then `docker compose --profile tunnel up -d --build`; return with `git checkout client-release`.
A database change must be undone	Restore the backup taken before the update (chapter 3.5), then build the previous version. Work synced after the backup must be re-sent or re-entered. Do this with the developer.

1.8 Never

- `docker compose down -v`, `docker volume rm`, `docker system prune --volumes`: they delete the database.
- Deleting anything under `C:\InspectionSystem\runtime`.
- Copying the live database files as a "backup" (use `Backup-All.ps1`).
- Publishing port 5432 or the API port.
- Running the stack on a second PC with the same tunnel token: Cloudflare would split traffic between two databases.

2. Accounts (Manager accounts and recovery)

2.1 Role names on screen and in the database

Sign-in card	Database role (`users.role`)	Created / reset by
Manager	`admin`	Office PC commands below (only way)
Admin	`supervisor`	A Manager, in the app (Technician List)
Technician	`inspector`	A Manager, in the app (Technician List)

The commands `createAdmin.js` and `resetAdminPassword.js` create and reset **Manager** accounts, despite their names. Usernames are case-sensitive and unique across all roles; passwords are 12–1024 characters. Every run is recorded in the audit log (`audit_events`) without the password.

Run these in PowerShell in `C:\InspectionSystem\app`, with the stack running. **Do not add `-T`**: the hidden password prompt needs a terminal. Nothing is shown while typing a password; that is normal.

2.2 Create a Manager

```
docker compose exec api node dist/cli/createAdmin.js new-manager-username
# optional display name:
docker compose exec -e ADMIN_DISPLAY_NAME="Full Name" api node dist/cli/createAdmin.js new-manager-username
```

Type the password twice. Expected: `Admin user 'new-manager-username' has been created.` It refuses a username that already exists (any role) and never changes an existing account. On a new installation this is the first step: the system ships with no accounts. Let the new Manager type their own password if possible.

2.3 Reset a Manager password (also re-activates the account)

```
docker compose exec api node dist/cli/resetAdminPassword.js manager-username
```

Type the new password twice. Expected: `Password for admin 'manager-username' has been reset; the account is active and N session(s) were signed out.` It refuses Technician/Admin usernames (reset those in the app).

2.4 Find Manager usernames

`<db user>` and `<db name>` are `POSTGRES_USER` and `POSTGRES_DB` in `.env` (normally `inspection_app` and `inspection`):

```
docker compose exec postgres psql -U <db user> -d <db name> -c "SELECT username, is_active FROM users WHERE role='admin';"
```

2.5 Retire a Manager who leaves

There is no button or command for this. Take a backup first, and make sure another active Manager can sign in.

1. Reset the leaving Manager's password to a long random value (2.3) and do not record it. This signs them out everywhere.
2. Mark the account inactive:

```
docker compose exec postgres psql -U <db user> -d <db name> -c "UPDATE users SET
is_active=false, updated_at=now() WHERE username='leaving-manager' AND role='admin';"
docker compose exec postgres psql -U <db user> -d <db name> -c "UPDATE user_sessions SET
revoked_at=now() WHERE revoked_at IS NULL AND user_id=(SELECT id FROM users WHERE
username='leaving-manager');"
```

3. Never `DELETE` the row: approvals, corrections and reports refer to it.

This SQL route is a manual procedure written from the login code; it is not covered by an automated test. To undo it, run the reset command (2.3), which re-activates the account.

2.6 Technician and Admin accounts

Managed by a Manager in the app (Technician List): add, rename, set new password, deactivate, reactivate, permanently remove (Technicians only), assign Admins. See the Client Handover Pack, chapter 2.

3. Backup and Recovery

3.1 What one backup set contains

`scripts\Backup-All.ps1` writes three files, each with a `.manifest.json` (SHA-256 fingerprint):

File (under `C:\InspectionSystem\runtime\backups\`)	Contains
`postgres\inspection-<date>-<time>.dump`	The whole database (`pg_dump` custom format)
`uploads\uploads-<date>-<time>.zip`	Every photo and uploaded file
`operational\operational-<date>-<time>.zip`	`company-profile.json`, the uploaded logo, `service-due-email.key`

It verifies each file against its manifest, copies the set to the off-PC folder (`-CopyTo`) and verifies the copies, deletes local sets older than `-KeepDays` (default 14, always keeping the newest `-KeepAtLeast` 3, and never touching the off-PC folder), appends to `backups\backup-log.txt`, and exits 1 on any failure.

Keep a copy of `.env` in the client's password manager as well (once, and after every change).

3.2 The daily scheduled task (set up once)

1. Test by hand:

   ```
   cd C:\InspectionSystem\app
   .\scripts\Backup-All.ps1 -CopyTo 'E:\InspectionBackups'
   ```

   The last line must be **Backup finished OK**.

2. Task Scheduler → **Create Task…**:

Tab	Setting
General	Name `Inspection PWA daily backup`. User: the Windows account that runs Docker Desktop. **Run only when user is logged on** (Docker Desktop runs in that session).
Triggers	Daily, at a time the PC is on and signed in (e.g. 18:30)
Actions	Program `powershell.exe` · Arguments `-NoProfile -ExecutionPolicy Bypass -File "C:\InspectionSystem\app\scripts\Backup-All.ps1" -CopyTo "E:\InspectionBackups"` · Start in `C:\InspectionSystem\app`
Conditions	Untick **Start the task only if the computer is on AC power**
Settings	Tick **Run task as soon as possible after a scheduled start is missed**; stop if it runs longer than 2 hours

3. Right-click the task → **Run**. **Last Run Result** must be `(0x0)` and the log must get a new **Backup finished OK** line. `(0x1)` = failed; the reason is the last line of `backup-log.txt`.

Off-PC place options: a cloud-synced folder the client owns (file history included), two BitLocker-encrypted USB drives swapped weekly, or a NAS share the Windows account can write to.

3.3 Retention

On the PC: 14 days (automatic). Off-PC: last 14 daily sets plus the first set of each month for 12 months, pruned by hand monthly. Each set includes all photos, so sets grow; check sizes and free disk space monthly.

3.4 Restore drill (every 3 months; never touches live data)

```
cd C:\InspectionSystem\app
$dump = (Get-ChildItem C:\InspectionSystem\runtime\backups\postgres\*.dump | Sort-Object
LastWriteTime | Select-Object -Last 1).FullName
.\scripts\Restore-Database.ps1 -BackupFile $dump
docker exec inspection-restore-drill psql -U inspection_app -d inspection -c "SELECT count(*) AS
users FROM users;"
docker exec inspection-restore-drill psql -U inspection_app -d inspection -c "SELECT count(*) AS
visits FROM inspection_jobs;"
docker rm -f inspection-restore-drill
$zip = (Get-ChildItem C:\InspectionSystem\runtime\backups\uploads\*.zip | Sort-Object LastWriteTime
| Select-Object -Last 1).FullName
.\scripts\Restore-Uploads.ps1 -ArchiveFile $zip
```

The counts should match the app. Photos land in `C:\InspectionSystem\runtime\restore-staging\uploads`; open a few, then delete that folder. Record the result in the Handover Forms (restore test log).

3.5 Rebuild after the PC dies, or restore a damaged database

Do this with the developer. Expect 2–3 hours on a new PC, mostly Docker installation.

1. Tell technicians **not to clear app data or uninstall**; unsent work waits on the phones.
2. **New PC:** install as on the first day (Docker Desktop, the code from the `client-release` branch, `Prepare-ClientPC.ps1`). In Cloudflare, give the new PC the tunnel and make sure the old PC can never reconnect. **Do not start the system yet. Existing PC with a damaged database:** `docker compose stop api reminder-worker`, then take a safety backup: `.\scripts\Backup-All.ps1`.
3. Copy the newest complete set from the off-PC place to `C:\InspectionSystem\restore-from\` and verify it:

   ```
   Get-ChildItem C:\InspectionSystem\restore-from -Recurse -Include *.dump,*.zip | ForEach-Object
   { .\scripts\Verify-Backup.ps1 -BackupPath $_.FullName }
   ```

4. Put back the operational folder and the photos (new PC: these folders are empty):

   ```
   Expand-Archive C:\InspectionSystem\restore-from\operational\operational-<date-time>.zip -
   DestinationPath C:\InspectionSystem\runtime\operational -Force
   Expand-Archive C:\InspectionSystem\restore-from\uploads\uploads-<date-time>.zip -DestinationPath
   C:\InspectionSystem\runtime\uploads -Force
   ```

5. Replace the database with the backup:

   ```
   docker compose up -d postgres
   docker compose ps postgres                          # wait for (healthy)
   docker compose cp C:\InspectionSystem\restore-from\postgres\inspection-<date-time>.dump
   postgres:/tmp/restore.dump
   docker compose exec -T postgres dropdb -U inspection_app --force inspection
   docker compose exec -T postgres createdb -U inspection_app inspection
   docker compose exec -T postgres pg_restore -U inspection_app -d inspection --no-owner --exit-on-
   error /tmp/restore.dump
   docker compose exec -T postgres rm /tmp/restore.dump
   ```

6. `docker compose --profile tunnel up -d --build`, then check: a Manager can sign in; customers and visits are there; a recent visit shows its photos; a PDF has the right letterhead and logo.
7. Work synced **after** the backup must be re-entered; work still waiting on phones uploads by itself.
8. Run `Backup-All.ps1` and confirm the scheduled task works on this PC.

Step 5 was rehearsed on a throwaway database on 2026-10-22 (restore with no error; user and visit counts matched the backup). Do the full sequence once on a spare PC as the first drill.

3.6 Never

- Never copy the live database files as a backup.
- Never keep the only backup on the same PC.
- Never e-mail a backup or put it in a chat: it holds all customer data and the e-mail key.

4. Security Details

Area	How it is done
Transport	HTTPS ends at Cloudflare; the tunnel to the PC is encrypted and outbound-only (no router ports opened). HSTS is sent for the real hostname.
Network	Docker networks: `postgres` and `api` sit on an internal-only network; only `proxy` is reachable; `cloudflared` can reach only `proxy`. Port 5432 is never published.
Headers	Strict Content-Security-Policy (no inline or third-party scripts), `X-Frame-Options: DENY`, `nosniff`, a `Permissions-Policy` that allows only the camera. Defined in `infra/reverse-proxy/Caddyfile`.
Cloudflare settings	Rocket Loader, Auto Minify, Email Obfuscation and Web Analytics **off** for the hostname (the CSP blocks injected scripts); a cache rule bypasses `/sw.js`, `/index.html`, `/manifest.webmanifest` and `/api/*`.
Sessions	Random token in an `HttpOnly`, `Secure`, `SameSite=Lax` cookie; server-side sessions; lifetime `SESSION_DURATION_HOURS` (default 12); a password reset revokes all sessions.
Passwords	Argon2id; 12–1024 characters; sign-in cooldown after repeated failures.
Authorisation	Every API route requires a session and checks role and ownership on the server. Technicians can open only their own visits.
Audit	Account changes, approvals and corrections are written to `audit_events`. Accepted inspections cannot be deleted.
Secrets	`.env` (database password, tunnel token) and `runtime\operational\service-due-email.key` are outside Git. The Gmail app password is encrypted at rest with that key; restoring a database without its key means re-entering the Gmail app password.
Phones	Unsynced work and downloaded reference data sit in the browser's storage for the app's origin. A record is marked synced only when the server confirms its exact ID; replays are idempotent.

If a secret leaks

Secret	Action
Tunnel token	Cloudflare Zero Trust → the tunnel → **Refresh token**; put the new token in `.env`; `docker compose --profile tunnel up -d cloudflared`
Manager password	Reset it (chapter 2.3)
Technician/Admin password	A Manager sets a new one in the app
Gmail app password	Revoke it in the Google account, create a new one, enter it in Settings → Email Reminders
Database password	The database is not reachable from outside; rotate it with the developer at the next visit